



COLLEGE OF
**ENGINEERING AND
COMPUTER SCIENCE**

ECE 520/L

System-on-Chip Design with Lab

Lab #6 Report
SPI Sensor SoC Integration
4/27/2026

Sean Felker
Instructor: Aaron Nanas
Fall 2025

Table of Contents

1	Introduction	2
2	Components.....	2
3	Hardware Development and Verification.....	3
4	Software Development and Integration	11
5	Conclusion	13
6	References.....	14

1 **Introduction**

In this lab, a Serial Peripheral Interface (SPI) master hardware module is implemented in Verilog. Its functionality is verified before it is packaged as a custom AXI-Lite peripheral IP block in Vivado. The IP block is connected to the Zynq Processing System (PS) using an AXI interconnect. The SPI connections are routed out to the PMOD pins of the Zybo Z7-10 ARM/FPGA SoC Development Board to connect with a PMOD Ambient Light Sensor (ALS). The hardware platform developed in Vivado is exported to the Vitis IDE and a C program is written to configure and control the SPI master IP block by interacting with its memory-mapped registers. The software does this to read the light sensor data from the PMOD ALS and print it out to the serial terminal on the host computer.

2 **Components**

Item	Quantity	Manufacturer
Zybo Z7-10 Zynq-7010 ARM/FPGA SoC Development Board	1	Diligent
USB-A to Micro USB Cable	1	N/A
PMOD Ambient Light Sensor	1	Diligent

Software Used:

- Vivado 2023.2
- Vitis IDE 2023.2

3 Hardware Development and Verification

In SPI communication, one device operates as the SPI master, while the other one or more connected devices operate as SPI slaves. There are 4 signals of communication. The Master Out Slave In (MOSI) is the bus for the SPI master to send data to the SPI slave. The Master In Slave Out (MISO) is the bus for the SPI slave to send data to the SPI master. The Chip Select (CS) signal driven by the master and is used to enable/activate a SPI transfer. The SPI clock (SCLK) pin is an oscillating clock signal driven by the master that is used to synchronize each bit of data that is transferred. For this lab, a SPI master is implemented using SPI mode 3, where data is sampled on the rising edge of SCLK and shifted out on the falling edge. SCLK idles high, CS is active low, bits are sent and received using most-significant bit (MSB) first, and transfers consist of 16 bits to match with the SPI settings of the PMOD ALS.

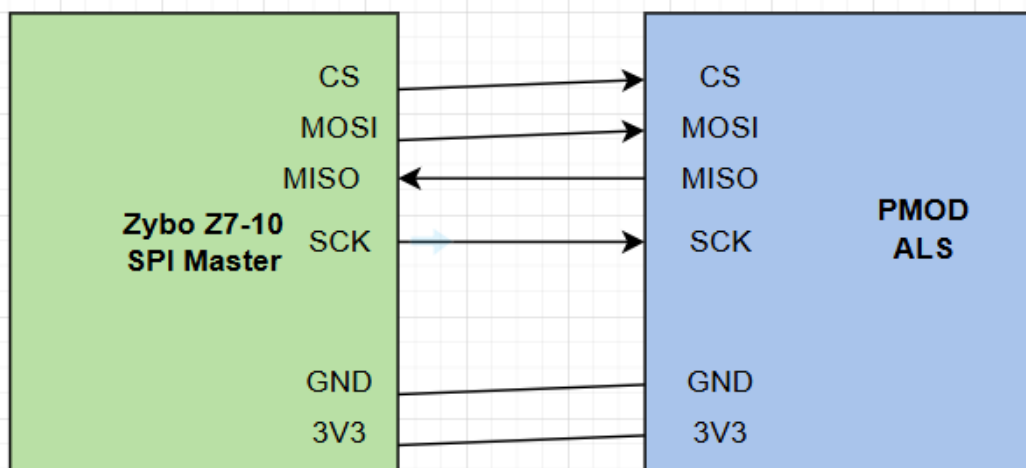


Figure 1: SPI pin connections between the Zybo Z7-10 and the PMOD ALS.

The first component designed for the SPI master module is the SPI clock divider. Given a clock divider value as an input, the module produces two pulse signals, 'fall_en' and 'rise_en', each respectively signaling a rising and falling edge of the SCLK. The module uses a counter to divide the input clock frequency following the equation $SCLK\ freq = \frac{sysclk}{(2*(clkdiv+1))}$. The PMOD ALS works with SCLK frequencies between 1 MHz and 4 MHz, so a clock divider value of 24 is used to get an SCLK frequency of 1 MHz.

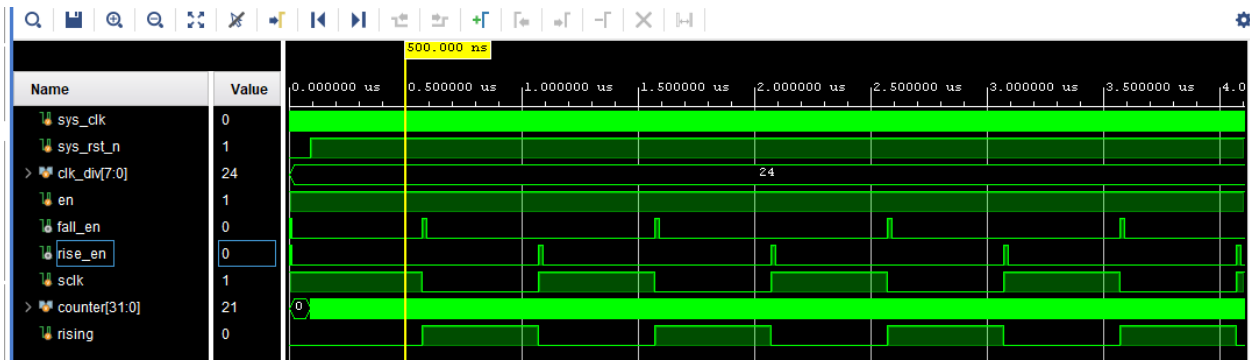


Figure 2: SPI clock divider module testbench waveform.

The SPI shift register module handles the serial data processing for the SPI module. On each SCLK falling edge, it shifts a bit of the data to be sent out to the MOSI signal one bit at a time, MSB first. On each SCLK rising edge, it samples a bit from the MISO line and stores it into the receive shift register. When the transfer completes (16 bits), the receive register is latched.

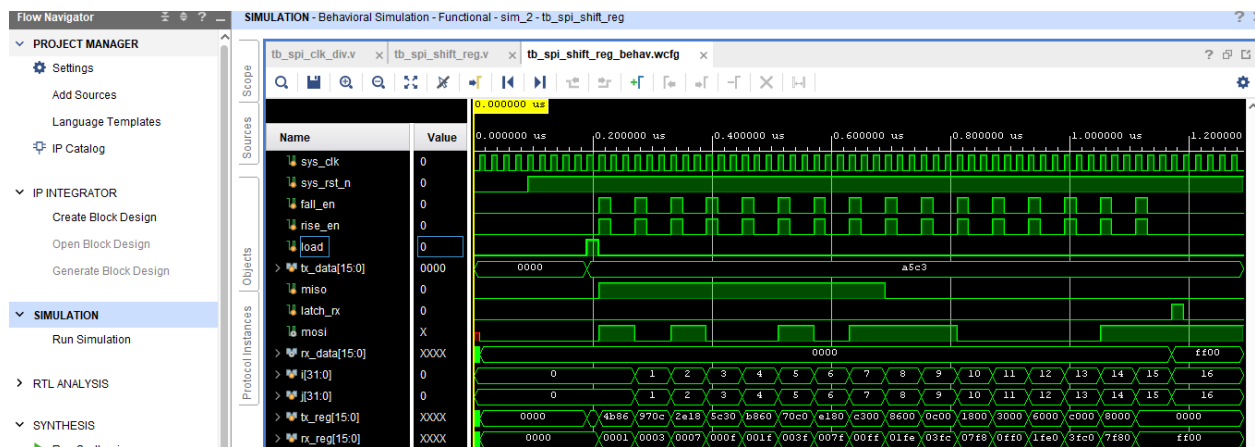


Figure 3: SPI shift register module testbench waveform.

The SPI CS control module controls the CS signal of the SPI module. Upon receiving an assertion signal, the module sets the CS output signal to active low until a deactivate signal is received, at which point the CS signal will be driven high.

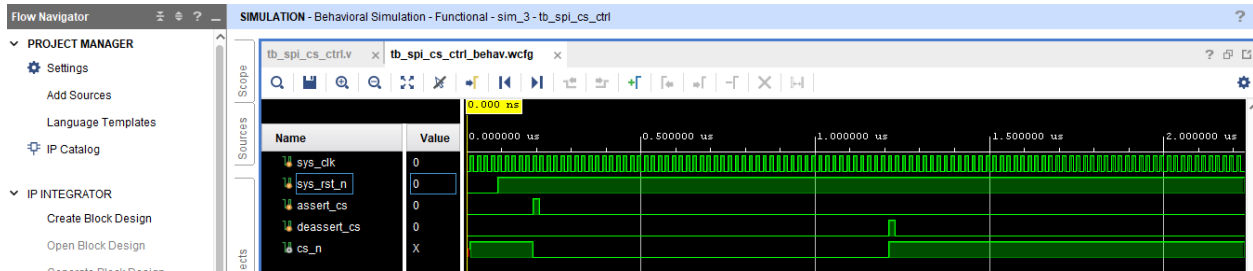


Figure 4: SPI CS control module testbench waveform.

The SPI finite state machine (FSM) module sequences the entire module through a full SPI transfer by driving control signals to the other three modules. In the idle state, the FSM waits until a start signal is received while the module is enabled. To start a SPI transfer, the FSM asserts the CS signal and loads the transmit data to the transmit shift register. In the transfer state, the SCLK is active and a counter is used to wait for 16 SCLK rising edges. After the 16th rising edge, the full SPI data has been received and is latched. In the done state, the CS and SCLK signals are put back in idle state, and the FSM will transition back to the idle state on the next clock cycle.

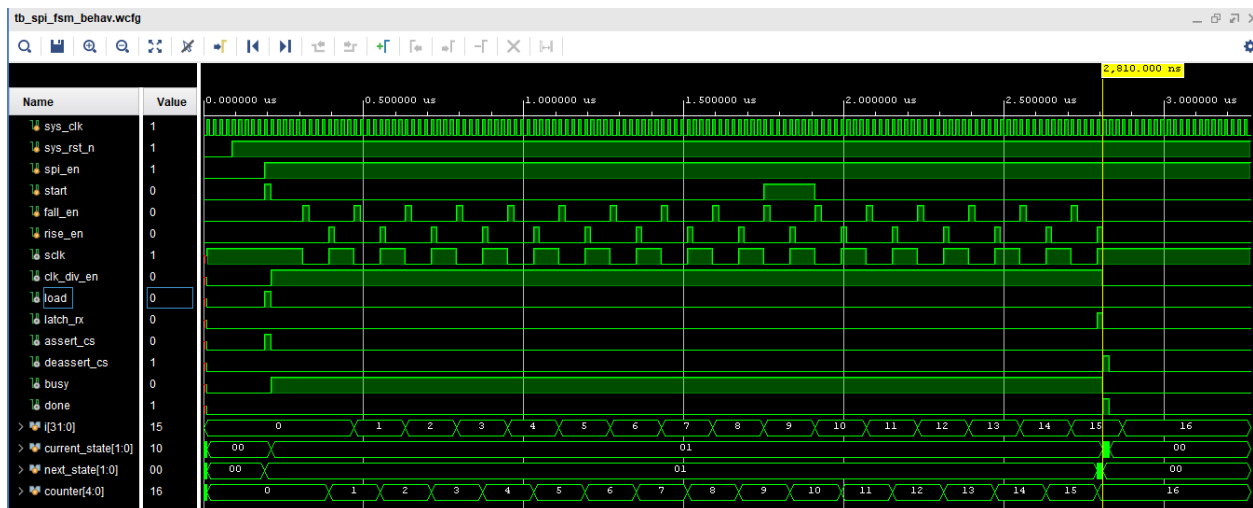


Figure 5: SPI FSM module testbench waveform.

The top-level module wires all internal signals of the submodules discussed above to build the full SPI master. The top-level SPI master is verified with a testbench to ensure all

components can function together to perform a full SPI transfer. The MISO line is manually driven by the testbench to ensure the SPI module can receive a full 16-bit data frame.

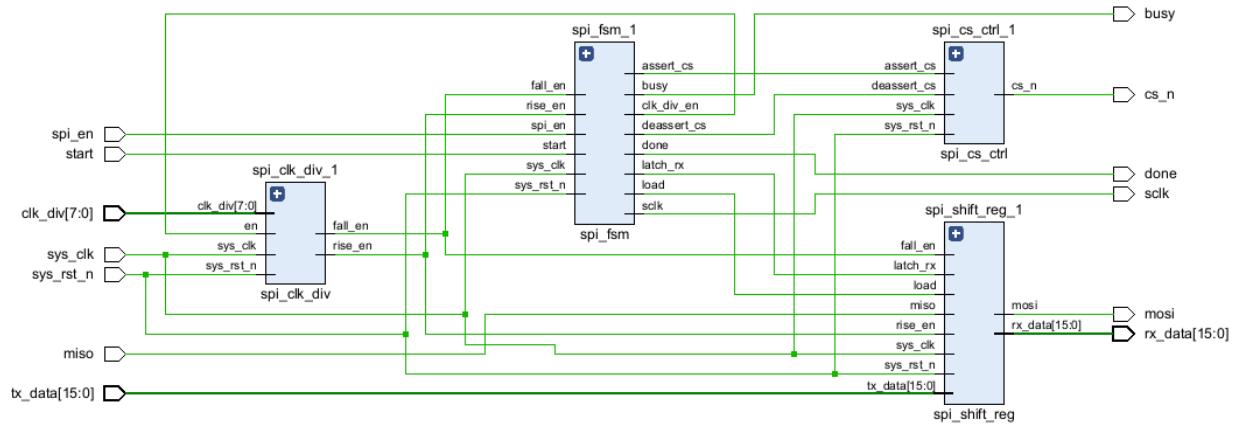


Figure 6: SPI top-level RTL schematic connecting the clock divider, FSM, CS control, and shift register submodules.

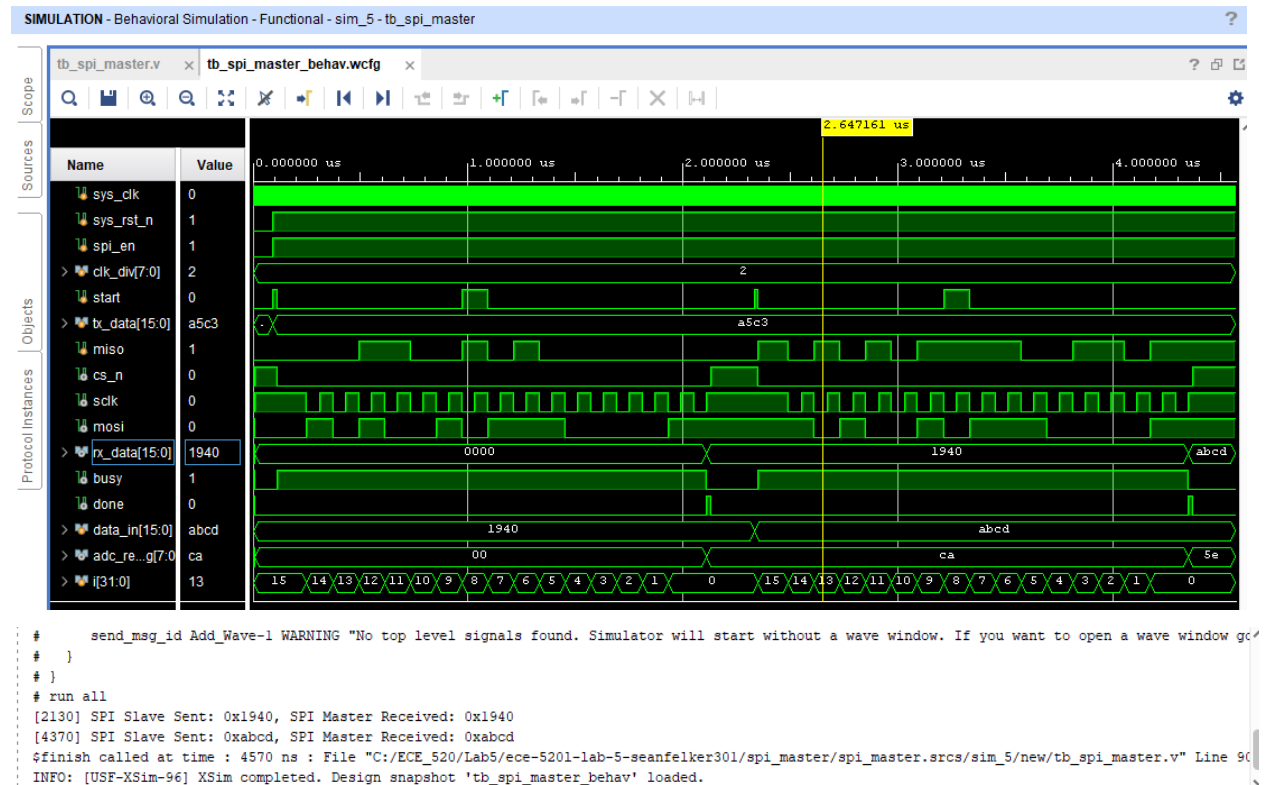
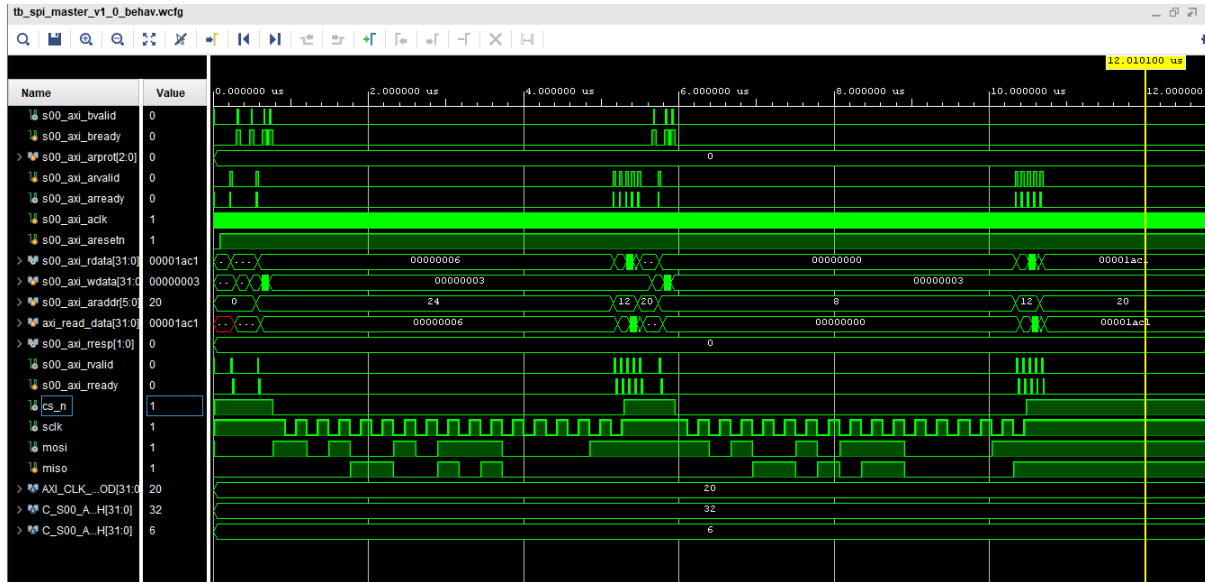


Figure 7: SPI top-level module testbench waveform and TCL console output showing the successful sending and receiving of a 16-bit data frame.

After verifying the functionality of the SPI module, it is packaged into a custom AXI-Lite IP block using tools provided by Vivado. Memory-mapped registers are configured to allow the SPI module to be integrated with the Zynq Processing System (PS). A testbench is written to verify that the SPI module can be fully controlled through writing to and reading from these memory-mapped registers using AXI. A transfer is started by writing 1 to the START bit in the SPI Control register, MISO data is manually driven by the testbench, and the Receive Data register is read after the SPI transfer has completed to verify that the correct data has been received.



```
# run all
[270] AXI_READ Addr: 0x00000000 | Value: 0x52000006
[350] AXI_WRITE Addr: 0x00000004 | Value: 0x00000001
[530] AXI_WRITE Addr: 0x00000018 | Value: 0x00000006
[610] AXI_READ Addr: 0x00000018 | Value: 0x00000006
[690] AXI_WRITE Addr: 0x00000010 | Value: 0x0000a5c3
[690] Drive 0x00001940 to the MISO line
[770] AXI_WRITE Addr: 0x00000008 | Value: 0x00000003
[5210] AXI_READ Addr: 0x0000000c | Value: 0x00000001
[5290] AXI_READ Addr: 0x0000000c | Value: 0x00000001
[5370] AXI_READ Addr: 0x0000000c | Value: 0x00000002
[5370] busy deasserted
[5450] AXI_READ Addr: 0x0000000c | Value: 0x00000000
[5530] AXI_READ Addr: 0x00000014 | Value: 0x00001940
[5710] AXI_WRITE Addr: 0x00000008 | Value: 0x00000001
[5790] AXI_READ Addr: 0x00000008 | Value: 0x00000000
[5870] AXI_WRITE Addr: 0x00000010 | Value: 0x0000a5c3
[5870] Drive 0x00001a01 to the MISO line
[5950] AXI_WRITE Addr: 0x00000008 | Value: 0x00000003
[10390] AXI_READ Addr: 0x0000000c | Value: 0x00000001
[10470] AXI_READ Addr: 0x0000000c | Value: 0x00000001
[10550] AXI_READ Addr: 0x0000000c | Value: 0x00000002
[10550] busy deasserted
[10630] AXI_READ Addr: 0x0000000c | Value: 0x00000000
[10710] AXI_READ Addr: 0x00000014 | Value: 0x00001a01
$finish called at time : 12810 ns : File "C:/ECE_520/L
```

Figure 8: SPI AXI-Lite IP testbench results, controlling the SPI module through AXI reads and writes. 0x1940 is driven to the MISO line. After the SPI transfer is completed, this data is read at the Receive Data register.

FPGA Revision Register: Offset 0x00

Field	Bits	Access	Reset Value	Description
FPGA_REV_VALUE	[31:0]	RO	0x5200_0006	FPGA Revision value. Can be read to confirm that the AXI bus is operating properly

Software Reset Register: Offset 0x04

Field	Bits	Access	Reset Value	Description
RSVD	[31:1]	RO	0x0000_0000	Reserved Bits
SW_RST	[0]	W1C	0	Software-controlled reset. Self clears after 1 clock cycle.

SPI Control Register: Offset 0x08

Field	Bits	Access	Reset Value	Description
RSVD	[31:2]	RO	0x0000_0000	Reserved Bits
SPI_EN	[1]	RW	0	SPI Controller enable.
START	[0]	W1C	0	Write 1 to begin a 16-bit SPI transfer. Self clears after 1 clock cycle

SPI Status Register: Offset 0x0C

Field	Bits	Access	Reset Value	Description
RSVD	[31:2]	RO	0x0000_0000	Reserved Bits
DONE	[1]	RO	0	Latches high when a SPI transfer has completed. Clears when read by software.
BUSY	[0]	RO	0	Reads 1 if the SPI module is in the middle of a transfer.

SPI Transmit Data Register: Offset 0x10

Field	Bits	Access	Reset Value	Description
RSVD	[31:16]	RO	0x0000	Reserved Bits
TX_DATA	[15:0]	RW	0x0000	Data to be transmitted out on the next SPI transfer through the MOSI bus..

SPI Receive Data Register: Offset 0x14

Field	Bits	Access	Reset Value	Description
RSVD	[31:16]	RO	0x0000	Reserved Bits
RX_DATA	[15:0]	RO	0x0000	Stores the last full 16-bit data received from the MISO bus.

SPI Clock Divider Register: Offset 0x18

Field	Bits	Access	Reset Value	Description
RSVD	[31:8]	RO	0x00_0000	Reserved Bits
CLK_DIV	[7:0]	RW	0x00	Clock divider value to generate the SCLK frequency following the formula, $SCLK\ freq = \frac{sysclk}{(2*(clkdiv+1))}$

Figure 9: Tables listing all of the SPI module’s memory-mapped registers and their functionalities.

The packaged AXI-Lite SPI master is connected to the Zynq7 Processing System using an AXI interconnect in Vivado’s block design tool. The MOSI signal is configured as an external input signal, and the CS, MOSI, and SCLK signals are configured as external outputs. These signals are connected to the Zybo board’s JA PMOD pins by configuring the XDC file for the Vivado project.

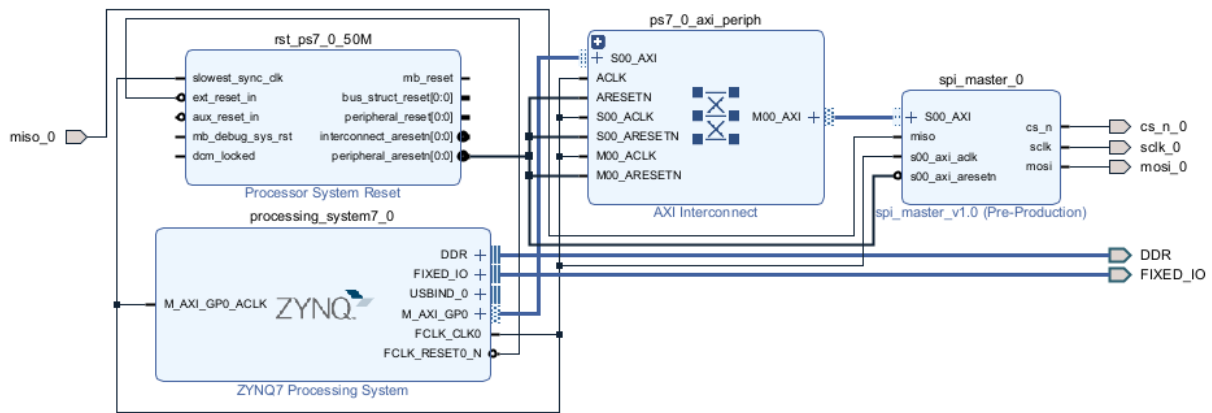


Figure 10: Vivado block diagram of the full SPI controller SoC.

4 Software Development and Integration

The integrated system design is exported as a hardware platform to the Vitis IDE. Software is written in C programming language for the Zynq PS to verify that the system can receive light sensor data from the connected PMOD ALS. The program implements an interactive menu controlled by the user's keyboard presses on the serial terminal of the connected host computer. For example, pressing '1' causes the SPI module to read light sensor data from the ALS and print it out. Other menu functions include enabling and disabling the SPI module, reading from and printing all memory-mapped registers, changing the clock divider value, initiating software reset, and performing 16 consecutive sensor readings and calculating the minimum, maximum, and the average of the light sensor data. To read the sensor data of the PMOD ALS, the software extracts bits 12 through 5 from the received 16-bit SPI data frame. The sensor outputs values from 0 to 255, with low values indicating low light level, and higher values indicating high light level.

```
h
----- ECE 520/L SPI Master test with PMOD Light Sensor - Sean Felker -----
----- SPI Controller Menu -----
'1': Perform a single SPI reading from the ALS
'2': Enable the SPI controller
'3': Disable the SPI controller
'4': Read and print SPI registers
'5': Change the CLK_DIV value
'6': Perform 16 SPI readings from the ALS
'h': Display the menu
'r': Software reset
-----
```

Figure 11: User menu options for the Vitis software to interact with the SPI module.

```

8-bit light level: 5
1
SPI Status Register: 0x00000001
SPI Status Register: 0x00000000
Raw 16-bit ALS data: 0x0300
8-bit light level: 24
1
SPI Status Register: 0x00000001
SPI Status Register: 0x00000000
Raw 16-bit ALS data: 0x1EC0
8-bit light level: 246
1
SPI Status Register: 0x00000001
SPI Status Register: 0x00000000
Raw 16-bit ALS data: 0x17E0
8-bit light level: 191
1
SPI Status Register: 0x00000001
SPI Status Register: 0x00000000
Raw 16-bit ALS data: 0x1F20
8-bit light level: 249

```

Figure 12: Pressing ‘1’ to perform single SPI transfers and printing out the received PMOD ALS data.

```

SPI controller disabled.
4
Printing SPI Registers-----
FPGA Revision Register: 0x52000006
Software Reset Register: 0x00000000
SPI Control Register: 0x00000000
SPI Status Register: 0x00000000
SPI TX Register: 0x00000000
SPI RX Register: 0x00001F20
SPI Clock Divider Register: 0x00000018
8-bit light level: 3
2
SPI controller enabled.
4
Printing SPI Registers-----
FPGA Revision Register: 0x52000006
Software Reset Register: 0x00000000
SPI Control Register: 0x00000002
SPI Status Register: 0x00000000
SPI TX Register: 0x00000000
SPI RX Register: 0x00000060
SPI Clock Divider Register: 0x00000018
8-bit light level: 3

```

Figure 13: Printing out the memory-mapped registers before and after enabling the SPI module.

```

5
Enter the clock divider value in hex: 4
SPI Clock Divider Register: 0x00000004
SCLK Frequency = 5000 kHz
5
Enter the clock divider value in hex: 18
SPI Clock Divider Register: 0x00000018
SCLK Frequency = 1000 kHz

```

Figure 14: Option to change the clock divider value for the SPI module. The new SCLK frequency is calculated and printed to the serial terminal.

```

Raw 16-bit ALS data: 0x1800
8-bit light level: 192
SPI Status Register: 0x00000001
SPI Status Register: 0x00000000
Raw 16-bit ALS data: 0x17A0
8-bit light level: 189
SPI Status Register: 0x00000001
SPI Status Register: 0x00000000
Raw 16-bit ALS data: 0x1620
8-bit light level: 177
SPI Status Register: 0x00000001
SPI Status Register: 0x00000000
Raw 16-bit ALS data: 0x1540
8-bit light level: 170
SPI Status Register: 0x00000001
SPI Status Register: 0x00000000
Raw 16-bit ALS data: 0x1740
8-bit light level: 186
SPI readings: 120, 121, 124, 126, 126, 132, 141, 163, 179, 187, 191, 192, 189,
177, 170, 186
Average = 157
Maximum = 192
Minimum = 120

```

Figure 15: Taking 16 consecutive ALS sensor readings. They are printed out in a list and the average, minimum, and maximum of the data is calculated.

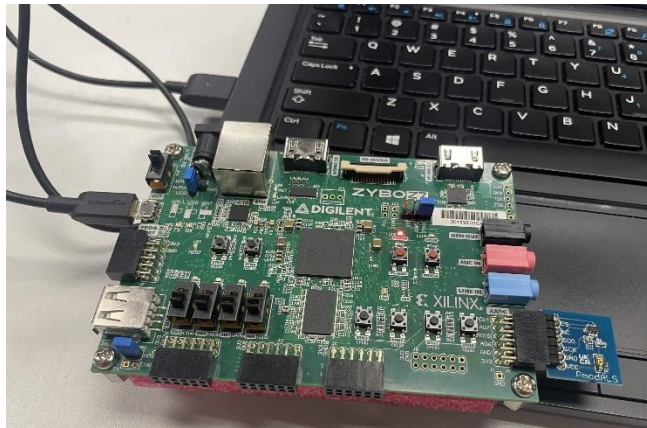


Figure 16: Picture of the Zybo Z7-10 board connected to the PMOD ALS and the host computer.

5 Conclusion

In this lab assignment, I learned how to implement and test a full SPI master module in a hardware description language. This helped me further understand the details of SPI communication, which I had previously only understood at a surface level. While debugging my SPI module, I learned the importance of breaking the project into smaller submodules, verifying each submodule, then combining them to build the full SPI controller. I further practiced packaging my hardware module into a custom AXI-Lite IP block and integrating it with the

Zynq PS. On the software side, I learned how to prompt the user for a 2-digit clock divider value in hexadecimal format using the 'isxdigit' and 'strtol' functions.

6 **References**

- <https://www.chipverify.com/verilog/verilog-syntax>
- <https://digilent.com/reference/pmod/pmodals/reference-manual>
- <https://www.chipverify.com/verilog/verilog-if-else-if>
- <https://fpgatutorial.com/how-to-write-a-basic-verilog-testbench/>
- <https://digilent.com/reference/programmable-logic/zybo-z7/reference-manual?srltid=AfmBOorqyKzhndboebVD-ra97dKznG82xplGYQVvDR9Oa5G2ylZgyxIy>
- <https://www.analog.com/en/resources/analog-dialogue/articles/introduction-to-spi-interface.html>
- <https://www.chipverify.com/systemverilog/systemverilog-fork-join>
- <https://www.asciitable.com/>
- <https://stackoverflow.com/questions/4638552/carriage-return-in-c>
- https://www.w3schools.com/c/ref_ctype_isxdigit.php
- <https://www.geeksforgeeks.org/cpp/strol-function-in-c/>